Implementing Hawkeye Replacement Policy in Champsim

The objective of this assignment is to implement the Hawkeye cache replacement policy in Champsim. It is based on the following paper.

A. Jain and C. Lin, “Back to the Future: Leveraging Belady’s Algorithm for Improved Cache Replacement,” ISCA 2016.

Deadline: 15th September, 2026.

See the "Deliverables" section at the end of this document for what to submit.

Baseline setup

1. Clone Champsim

```
git clone https://github.com/ChampSim/ChampSim.git
```

2. Download dependencies following steps in https://github.com/ChampSim/ChampSim. Install curl if needed - `sudo apt-get install curl zip unzip tar`.
3. Compile ChampSim using the default configuration script - `champsim_config.json`. Again following instructions from the github link.
4. Make a traces folder (inside the ChampSim directory)

```
mkdir -p traces && cd traces
```

5. Download traces from `https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/`. Do not decompress the .xz file as ChampSim reads it directly. For example:

```
wget https://dpc3.compas.cs.stonybrook.edu/champsim-
traces/speccpu/400.perlbench-41B.champsimtrace.xz
```

6. Run the baseline

```
bin/champsim --warmup_instructions 20000000 --simulation_instructions
50000000 traces/400.perlbench-41B.champsimtrace.xz
```

Here `warmup_instructions` specify the length of the warmup period. After the warmup period is over, the IPC statistics are reset, and the final reported IPC metric will be calculated starting at this point. Default value is 1,000,000. `simulation_instructions` specify how many instructions you want to

execute after the warmup period is over. After the simulation period is over, the simulator will exit and IPC since the warmup period will be printed. Default value is 10,000,000.

Hawkeye implementation

The Hawkeye replacement policy applies Belady's algorithm (referred to as OPT in the paper) to the past access stream and learns from it. The key insight is that if past behaviour predicts future behaviour, a policy that mimics OPT on history will approach OPT going forward. You have to develop the following components and tie them together for the complete implementation.

Architecture and file layout

The three algorithmic pieces below (OPTgen, the Hawkeye predictor, RRIP-based insertion/aging) must each be implemented as a **standalone component with no dependency on ChampSim types** (no `modules.h`, no ChampSim `BLOCK`/ `champsim::address` types, etc.), each in its own header/source pair. The ChampSim-facing replacement policy (`hawkeye.h`/ `hawkeye.cc`, the `struct hawkeye : public champsim::modules::replacement`) is a separate, thin **adapter** that owns one instance of each component and implements ChampSim's required hooks (`find_victim`, `replacement_cache_fill`, `update_replacement_state`).

We will also define individual test drivers for each of 3 pieces in a sibling directory at the repository root, `hawkeye_tests/`, which ChampSim's build never looks at. The test drivers will `#include` the component headers from `replacement/hawkeye/` directly (via an `-I` flag), so no code is duplicated.

```
ChampSim/
  replacement/
    hawkeye/
      optgen.h
      optgen.cc
      predictor.h
      predictor.cc
      rrip.h
      rrip.cc
      hawkeye.h
      hawkeye.cc
  hawkeye_tests/
    optgen_test.cc
    predictor_test.cc
    rrip_test.cc
```

Each section below tells you exactly what should be included in each of the files. Create this directory structure before starting on any of the components below:

```
mkdir -p replacement/hawkeye
mkdir -p hawkeye_tests
```

OPTGen - reconstructing OPT for the past (30 points)

The first step is reconstructing OPT for past accesses. For each reference to a line X and its next reuse X', OPTgen should answer would this access would have been a hit under OPT? Use the following three insights from the paper:

1. OPT's decision depends not just on a line's reuse interval but on the overlap of reuse intervals, i.e., the demand on the cache.
2. OPT's decision for a past access can be finalized at the time of its next use.
3. A history window of 8× the cache size is sufficient to reproduce OPT accurately.

You should implement the basic OPTgen described in Section 3.1 of the paper. You need to create and maintain an occupancy vector for each cache set as OPT makes independent replacement decisions within each set. For a W-way set-associative cache, each vector should contain 8W entries, representing the most recent 8W accesses mapped to that set. Each entry counts how many liveness intervals overlap at that time. For any access to address X, you need to examine its usage interval (from the previous access up to, but excluding, the current one). If every entry in that interval is below the associativity, W, OPT would have cached the line. Accordingly mark it a hit and increment those entries; otherwise it is a miss and the vector is left unchanged (a miss bypasses).

Note: You are **not required** to implement Set Dueling (Section 3.2.2). OPTgen should track every cache set passed to it via `num_sets`; you do not need to reduce this to a sampled subset of sets.

Implementation requirements

To make this component gradable independently of the full ChampSim build, you must implement OPTgen as a **standalone class** (no ChampSim types in its interface) with exactly the following signature, in `replacement/hawkeye/optgen.h` and `replacement/hawkeye/optgen.cc` (see "Architecture and file layout" above).

```
class OPTgen {
public:
    // num_sets: number of cache sets tracked independently
    // associativity: W, the cache associativity (occupancy vector cap)
    // history_multiplier: length of tracked history, in units of the set's
    //                      capacity (paper uses 8x; see Figure 2).
Default 8.
    OPTgen(std::size_t num_sets, std::size_t associativity,
           std::size_t history_multiplier = 8);

    // Processes one access to `address`, mapped to set `set_idx`, per
Section 3.1.
    bool access(std::size_t set_idx, uint64_t address);
};
```

The function signatures must be exact as the test driver (described next) will call them directly but you can (and will likely need to) add new structures, helper functions, or private members of your own to accomplish the overall functionality.

You must also provide a standalone test driver at `replacement/hawkeye/test/optgen_test.cc` that matches **exactly** the skeleton below — do not change the constructor call, the loop, or the print

statements. The only thing left empty is on purpose: we will substitute our own held-out `accesses` vector between the `TEST_VECTOR_START`/ `TEST_VECTOR_END` markers when grading, so the surrounding code must compile and run unmodified with an arbitrary vector of `(set_idx, address)` pairs.

```
// optgen_test.cc
#include "../replacement/hawkeye/optgen.h"
#include <iostream>
#include <vector>
#include <utility>
#include <cstdint>

int main() {
    OPTgen opt(/*num_sets=*/1, /*associativity=*/2);

    // TEST_VECTOR_START
    std::vector<std::pair<std::size_t, uint64_t>> accesses = {
    // left empty; the grading script substitutes its own (set_idx,
address) pairs here
    };
    // TEST_VECTOR_END

    int hits = 0;
    for (auto& [set_idx, addr] : accesses) {
        bool hit = opt.access(set_idx, addr);
        std::cout << std::hex << addr << std::dec << ": " << (hit ? "HIT" :
"MISS") << "\n";
        if (hit) hits++;
    }
    std::cout << "TOTAL HITS: " << hits << "\n";
}
```

This file must compile on its own without requiring the ChampSim build.

Hawkeye predictor - learning per-PC behavior (20 points)

Next, you should implement the predictor described in Section 3.3 of the paper. You need to implement a PC-indexed predictor (a table of small 3 bit saturating counters indexed by a hash of the load PC) that records whether accesses by a given instruction tend to be OPT-hits or OPT-misses. Each OPT label from OPTgen should train the counter for the corresponding PC by incrementing toward “cache-friendly” on an OPT hit and decrementing toward “cache-averse” on an OPT miss. When inserting a new cacheline, the predictor should be queried with the current PC to classify the line as cache-friendly or cache-averse. The higher order bit of the counter should indicate whether the line is cache-friendly (1) or averse (0);

Implementation requirements

Similar to OPTgen, implement the predictor as a **standalone class**, independent of ChampSim types, with the following signature, in `replacement/hawkeye/predictor.h` and `replacement/hawkeye/predictor.cc` (see "Architecture and file layout" above):

```
class HawkeyePredictor {
public:
    // num_entries:  size of the PC-indexed table (paper: 8K entries)
    // counter_bits: width of the saturating counter (paper: 3 bits, range
[0, 2^counter_bits - 1])
    HawkeyePredictor(std::size_t num_entries = 8192, int counter_bits = 3);

    // Trains the counter indexed by a hash of `pc` per Section 3.3's
update rule.
    void train(uint64_t pc, bool opt_hit);

    // Returns the predicted classification for `pc`.
    bool predict(uint64_t pc) const;

    // Debug-only accessor: raw counter value in [0, 2^counter_bits - 1].
    int get_counter(uint64_t pc) const;
};
```

The function signatures must be exact as the test drivers (described next) will call them directly but you can (and will likely need to) add new structures, helper functions, or private members of your own to accomplish the overall functionality.

You must also provide a standalone test driver at `replacement/hawkeye/test/predictor_test.cc` that matches **exactly** the skeleton below — do not change the construction, the loops, or the print statement. Only the two vectors between the `TEST_VECTOR_START`/ `TEST_VECTOR_END` markers are left empty on purpose: we will substitute our own held-out training events and query PCs when grading, so the surrounding code must compile and run unmodified with arbitrary vectors of that shape.

```
// predictor_test.cc
#include "../replacement/hawkeye/predictor.h"
#include <iostream>
#include <vector>
#include <utility>
#include <cstdint>

int main() {
    HawkeyePredictor pred;

    // TEST_VECTOR_START
    // train_events: ordered (pc, opt_hit) pairs applied via pred.train(pc,
opt_hit)
    std::vector<std::pair<uint64_t, bool>> train_events = {
    };
    // query_pcs: pcs to print counter/prediction for, after all training
is applied
    std::vector<uint64_t> query_pcs = {
    };
    // TEST_VECTOR_END
```

```
    for (auto& [pc, opt_hit] : train_events) {
        pred.train(pc, opt_hit);
    }
    for (uint64_t pc : query_pcs) {
        std::cout << std::hex << pc << std::dec
                  << ": counter=" << pred.get_counter(pc)
                  << " predict=" << pred.predict(pc) << "\n";
    }
}
```

This file must again compile independently of the ChampSim build.

RRIP-based insertion and aging (10 points)

You need to also store an RRIP value per cached line and use it to choose victims. The following values should be used for every new cacheline insertion • Cache-friendly insertion -> Set highest priority (RRPV = 0). • Cache-averse insertion -> Set lowest priority (e.g. RRPV = 7), making it the immediate eviction candidate. Whenever you need to evict a cacheline, prefer any line with the maximum RRPV. If none exists, age lines (increment RRPVs) as in RRIP so the policy adapts to phase changes.

Implementation requirements

Implement the update and victim-selection logic as **standalone functions**, independent of ChampSim types, with the following signature, in `replacement/hawkeye/rrip.h` and `replacement/hawkeye/rrip.cc` (see "Architecture and file layout" above):

```
enum class Classification { CACHE_FRIENDLY, CACHE_AVERSE };

// Applies Table 1's update rule to each set's RRPV vector.
void update_rrpv(std::vector<int>& rrpv, std::size_t way,
                 Classification cls, bool is_hit);

// Selects a victim way, aging the set if necessary, per Section 3.4.
std::size_t find_victim(std::vector<int>& rrpv);
```

The function signatures must be exact as the test drivers (described next) will call them directly but you can (and will likely need to) add new structures, helper functions, or private members of your own to accomplish the overall functionality.

You must also provide a standalone test driver at `replacement/hawkeye/test/rrip_test.cc` that matches **exactly** the skeleton below — do not change the `RRIPOp` struct, the loop, or the print statements. Only the two vectors between the `TEST_VECTOR_START`/`TEST_VECTOR_END` markers are left empty on purpose: we will substitute our own held-out initial RRPV state and operation sequence when grading, so the surrounding code must compile and run unmodified with an arbitrary starting state and arbitrary ops.

```cpp
// rrip_test.cc
#include "../replacement/hawkeye/rrip.h"
#include <iostream>
#include <vector>
#include <cstddef>

int main()
{
  // --- Case 1: insertion policy ---
  std::vector<int> rrpv = {
  // left empty; the grading script substitutes initial per-way RRPV state
  };
  update_rrpv(rrpv, 0, Classification::CACHE_AVERSE, /*is_hit=*/false);
  update_rrpv(rrpv, 1, Classification::CACHE_FRIENDLY, /*is_hit=*/false);
  for (int v : rrpv)
    std::cout << v << " ";
  std::cout << "\n";
  std::cout << "victim: " << find_victim(rrpv) << "\n";

  // --- Case 2: find a victim ---
  std::vector<int> rrpv2 = {
  // left empty; the grading script substitutes initial per-way RRPV state
  };
  std::size_t v = find_victim(rrpv2);
  for (int x : rrpv2)
    std::cout << x << " ";
  std::cout << "\n";
  std::cout << "victim: " << v << "\n";
}
```

This file must compile independently of the ChampSim build.

Important — We will use automated grading: We will grade `optgen_test`, `predictor_test`, and `rrip_test` by comparing their exact stdout against the expected output. Any print statements that you add anywhere else in the code while debugging **should be removed before you submit**. A single stray line of debug print can make a submission fail the automated diff.

Complete integration and testing

1. Once `optgen.h`/ `optgen.cc`, `predictor.h`/ `predictor.cc`, and `rrip.h`/ `rrip.cc` are complete and passing their standalone tests, create `hawkeye.h`/ `hawkeye.cc`, which adapt them to ChampSim's replacement-module interface. A skeleton is given below, with some marked TODOs.

```cpp
// hawkeye.h
#ifndef HAWKEYE_H
#define HAWKEYE_H
#include <vector>
#include "modules.h"
#include "optgen.h"
#include "predictor.h"
```

```
#include "rrip.h"

struct hawkeye : public champsim::modules::replacement {
    // TODO: instantiate different modules

    // TODO: Add any new data structures or functions to connect each of
the modules

    // TODO: Complete the definitions for the following functions that are
required across all replacement policies. You can use the other replacement
policies as a reference. Each should be implemented primarily by calling
optgen.access(...), predictor.train(...)/predict(...), and
update_rrpv(...)/find_victim(...) from rrip.h, do not re-implement
OPTgen/predictor/RRIP logic here.
    //   find_victim (args);
    //   replacement_cache_fill (args);
    //   update_replacement_state (args);
};
#endif
```

```
// hawkeye.cc
#include "hawkeye.h"
#include <algorithm>

// TODO: implement find_victim / replacement_cache_fill /
update_replacement_state here, delegating to optgen / predictor / rrip.h as
described in hawkeye.h above.
```

2. Specify your policy for LLC in a config file (`hawkeye_config.json`):

```
{
  "LLC": { "replacement": "hawkeye" }
}
```

Note that the above is sufficient and we dont have to copy over the other configurations from `champsim_config.json` because Champsim already uses the default configurations for anything that is not specified.

3. Reconfigure and rebuild:

```
./config.sh hawkeye_config.json
make -j$(nproc)
```

4. Re-run the same benchmark used in baseline to sanity check.

Required experiments and plots (30 points)

For both experiments below, run both `"replacement": "lru"` and `"replacement": "hawkeye"` for the LLC and compare miss rate. Use `--warmup_instructions 20000000 --simulation_instructions 50000000` for every run in this section (20M warmup / 50M simulation instructions) to keep total runtime manageable given the number of runs required.

Plot 1: LLC miss rate vs. LLC associativity, on `456.hmmer-191B.champsimtrace.xz`

Sweep LLC associativity across 4, 8, and 16 ways, holding total LLC capacity fixed at 2MB (the ChampSim default) by adjusting the number of sets inversely (assuming the default 64B line size):

Associativity (ways)	Sets (for a fixed 2MB LLC, 64B line)
4	8192
8	4096
16 (default)	2048

For each of these 4 configurations, set the LLC's set/way fields in your config JSON (check your local `champsim_config.json` and the ChampSim docs for the exact key names), rebuild, and run both LRU and Hawkeye on `456.hmmer-191B.champsimtrace.xz` with the exact commands below (8 runs total). Plot LLC miss rate (%) on the y-axis against associativity on the x-axis, with one line for LRU and one for Hawkeye.

Each command must be run against the ChampSim binary rebuilt (`./config.sh ...` + `make -j$(nproc)`) with that row's `sets`/`ways` and `replacement` policy in the config as the command line itself does not select associativity or policy.

Plot 2: LLC miss-rate reduction over LRU, across benchmarks (default LLC: 2MB, 16-way)

Using the default LLC configuration (no associativity/size changes from your baseline), run both LRU and Hawkeye on each of the following three traces (6 runs total) with the same warmup and simulation instructions as before:

- `456.hmmer-191B.champsimtrace.xz` (reused from Plot 1)
- `429.mcf-22B.champsimtrace.xz`
- `473.astar-42B.champsimtrace.xz`

Each command must be run against the ChampSim binary rebuilt with the default LLC config (`sets=2048`, `ways=16`) and that row's `replacement` policy.

For each benchmark, compute `(missrate_LRU - missrate_Hawkeye) / missrate_LRU * 100` and plot this percentage as a bar chart, one bar per benchmark.

Deliverables

Submit the following:

1. Zipped code, or a link to your updated ChampSim repository hosted on GitHub.
2. A report (PDF) containing:

1. Top 3 learnings/observations from your implementation and experiments (10 points).
2. Two plots (mentioned in "Required experiments and plots" above):
    1. LLC miss rate vs. LLC associativity (4/8/16-way) for LRU and Hawkeye, on `456.hmmer-191B`.
    2. LLC miss-rate reduction (%) over LRU for Hawkeye, across `456.hmmer-191B`, `429.mcf-22B`, and `473.astar-42B`.

Reminder before you submit: Make sure you remove every debug print you added anywhere in your code beyond what each test driver's skeleton specifies. Grading is automated and diffs exact output, so stray prints can cause an otherwise-correct submission to fail.

Useful links

1. https://github.com/ChampSim/ChampSim/discussions/
2. https://deepwiki.com/maccoymerrell/ChampSimFix/1-overview